IS 2202 : ADVANCED DATA STRUCTURES AND ALGORITHMS

2. INTRODUCTION TO B-TREES

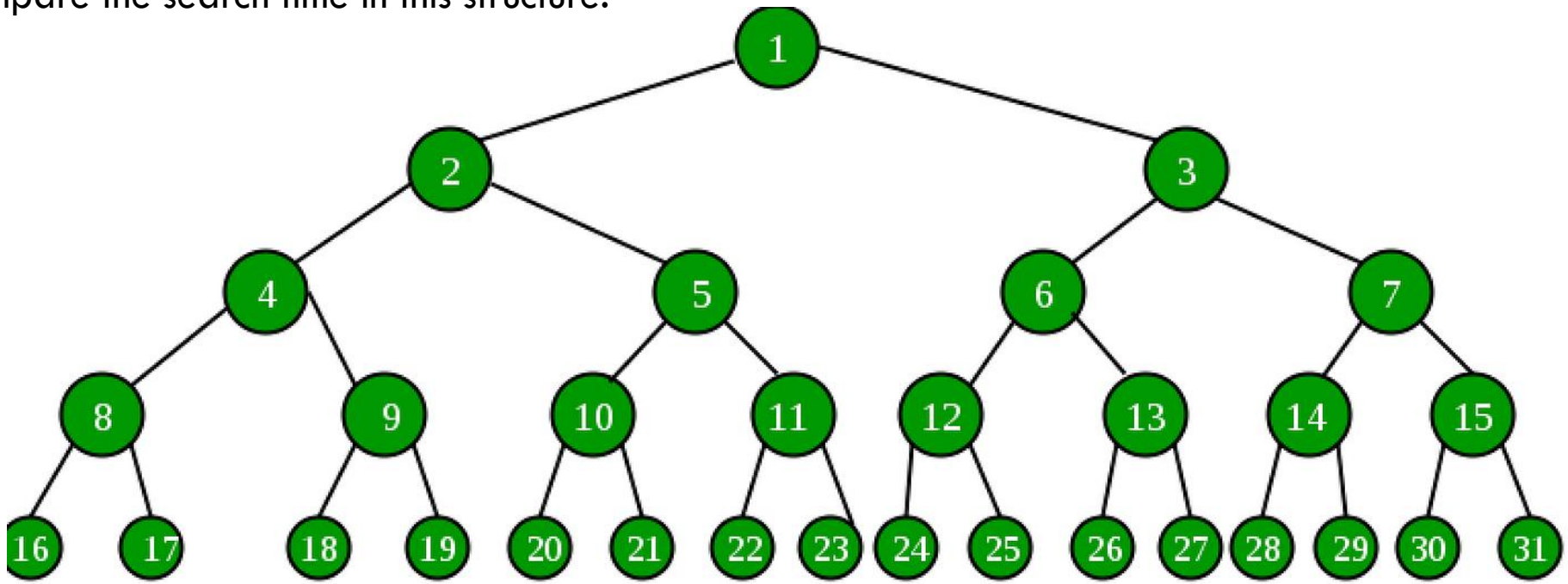
Dr. Yohani Ranasinghe
University of Colombo School of Computing

- B-Tree
- Search
- Insertion
- Deletion
- Complexity analysis

BINARY TREE

Search 21 ?

Compare the search time in this structure.

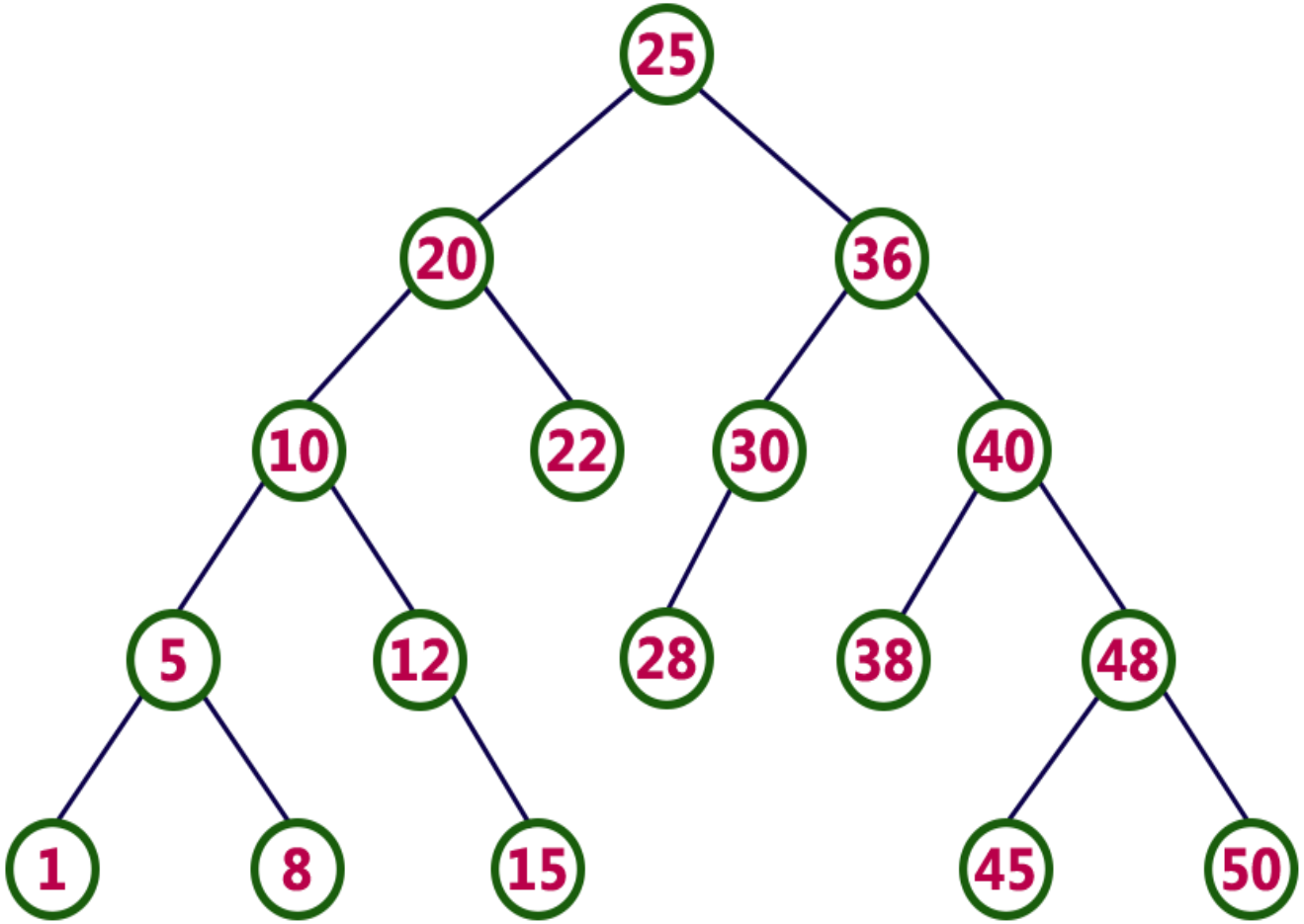


Operation	**Best Case**	**Worst Case**
Search	$O(\log n)$	$O(n)$

UCSC

BST

Find 21



1. Access Data from Memory
2. Comparison

The Hidden Bottleneck: Memory vs. Disk

⚡ RAM & CPU

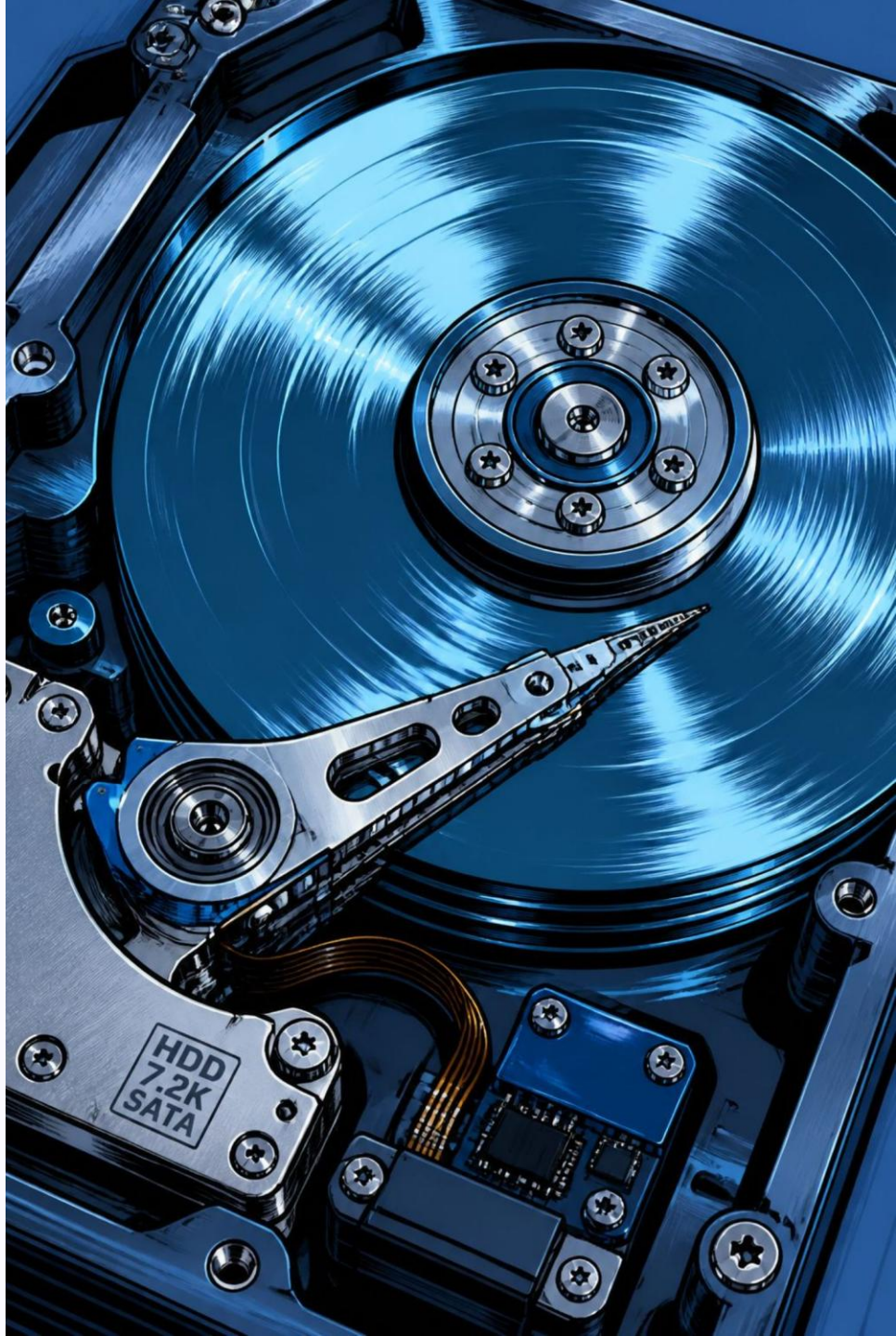
Nanosecond-scale operations. Lightning-fast, but limited in capacity.

🐢 Disk I/O

Millisecond-scale access. The dominant bottleneck for large datasets.

🌳 Binary Search Trees

Too many cache misses on disk. Each level may trigger a costly probe.

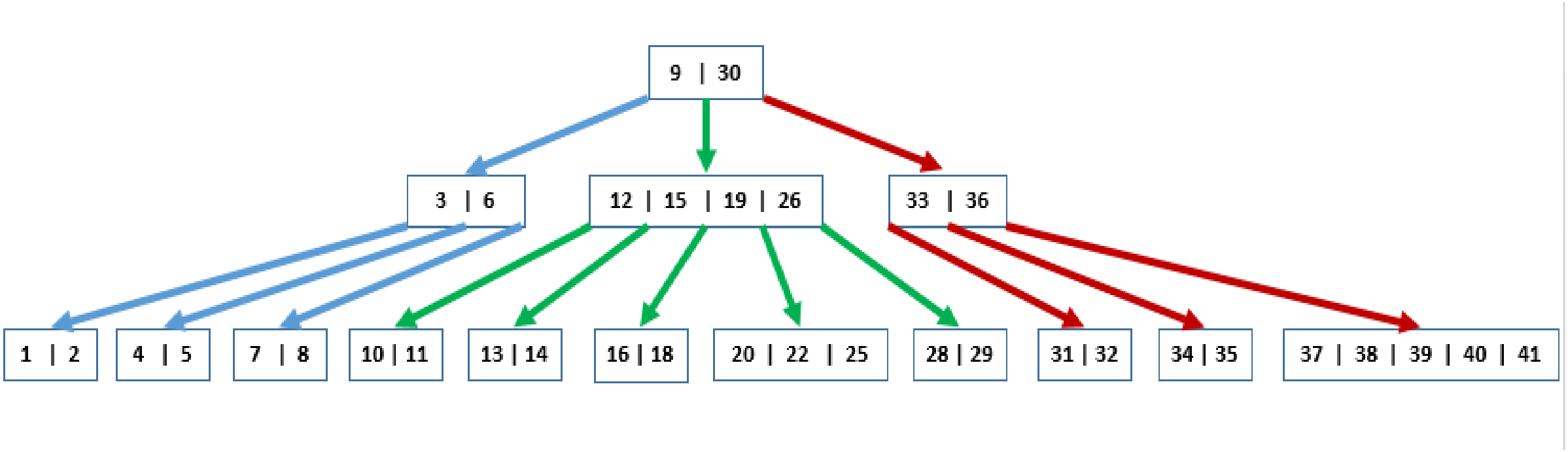


multiway search trees

What if a node could store multiple keys instead of one?

B Tree

B-TREE



A B-Tree is a specialized **m-way tree** designed to **optimize data access**, especially on **disk-based storage systems**

UCSC

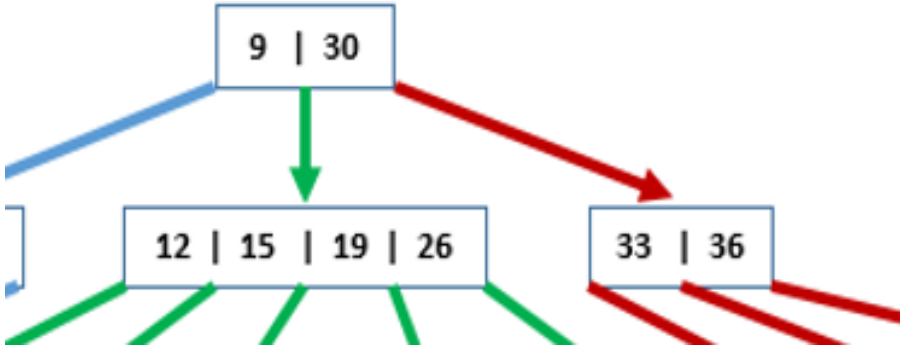
B-TREE

A B-Tree is a **self-balancing** search tree data structure that maintains sorted data and allows **efficient insertion, deletion, and search operations** in logarithmic time.

B-trees can have many children, reducing the height of the tree and improving search efficiency.

Optimized for systems that read and write large blocks of data, such as databases and file systems.

B-TREE PROPERTIES



1. **Nodes can have multiple keys and children:**
   - Each node can have **at most *m* children**
   - Each node can have at most m − 1 keys.
   - All keys in a node are **sorted**. This sorting occurs within each node and across the tree.

2. **Balanced structure:**
   - The tree is **height-balanced** — all leaf nodes are at the **same depth**.

3. **Key and child distribution:**
   - A non-leaf node with *k* keys has **exactly *k + 1* children**.
   - Each internal node (except the root) has at least [m/2] children.
   - The root node is an exception in terms of the minimum number of children; it must have at least two children if it is not a leaf node.

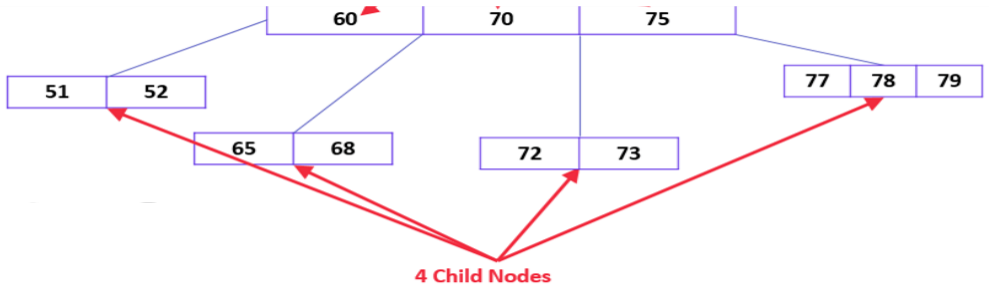
4. **Efficient operations:**
   - Search, insert, and delete operations take **O(log n)** time.
   - Suitable for **disk-based storage systems** due to reduced number of reads.

UCSC

ORDER OF A B-TREE

- The order, often represented by 'm', is the maximum number of children a node can have in the tree.
- Decided based on disk block and key sizes.
- Able to store a significant number of keys within a single node, including large key values.
- A higher order means that each node can contain more children, which usually results in a wider, shallower tree. A higher order typically results in a shorter tree, hence reducing costly disk operations

Eg: m=4, means a node can have 4 child nodes



EXAMPLE

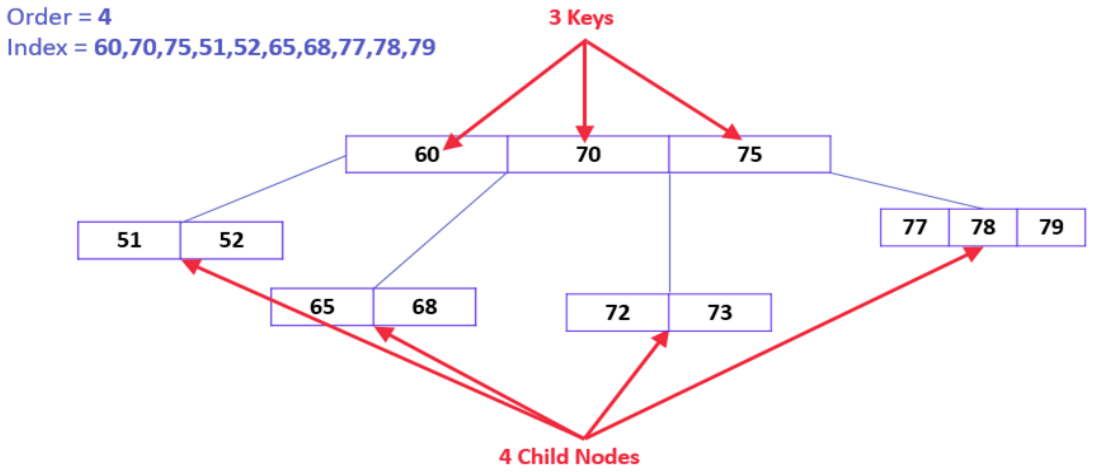
- B-Tree of Order m = 5, find the number of children.

Node Type	Min Children	Max Children
Root		
Internal Node		
Leaf Node		

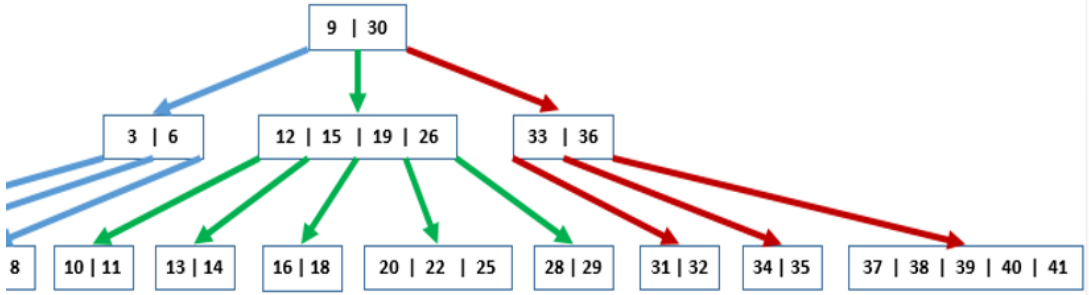
Node Type	Min Children	Max Children
Root	0/2	5
Internal Node	3 ([5⁄2])	5
Leaf Node	0	0

KEYS IN A B-TREE

- Keys are the values stored within the nodes of a B-tree.
- In a B-tree of order m, each node can have a maximum of m−1 keys.
- In a B-tree, keys serve as separators that define ranges for their child nodes. For example, if a node has keys K1 and K2(where K1<K2) then it will have three children: the left child contains values less than K1, the middle value will contain a value between K1 and K2 and the right child contains values greater than K2. Therefore B-tree maintains its data in sorted order.
- **Eg: m=4, means maximum number of keys = m-1=3**



INTERNAL NODES IN A B-TREE



- The root node must have no less than two children (0 in special cases).
- Each internal node (neither root nor leaf) must have minimum 'ceiling[m/2]' children. This ensures that the tree remains balanced and dense enough for efficient operations.

Eg: if we have a B-tree of order 4 (m = 4), then each internal node must have at least '[4/2] = 2' children. This means that even after some deletions that may cause children to be removed from a node, the tree must restructure itself (via merges or redistributing keys and children with adjacent nodes) to ensure that each internal node still has at least 2 children.

Question: What is the minimum number of keys?

LEAF NODES IN A B-TREE

- In B-trees, all **leaf nodes reside at the same level.**
- The path from the root node to any leaf node is of the same length.

Why Is This Important?

- When searching for a key, the search operation will always take a path from the root to a leaf. If all leaves are at the same level, every search operation has a predictable maximum number of steps, ensuring that the time complexity for search operations is always logarithmic in the number of keys O(log n)
- Efficient Searches
- Equal Access Time.

EXAMPLE

- B-Tree of Order m = 5, find the number of keys.

Node Type	Min Keys	Max Keys
Root		
Internal Node		
Leaf Node		

Node Type	Min Keys	Max Keys
Root	1	4
Internal Node	([5⁄2])-1 =2	4
Leaf Node	2	4

HOW IS THE ORDER DETERMINED?

Disk block (page) size = **4096 bytes (4 KB)**

Each key = **8 bytes**

Each child pointer = **8 bytes**

For a B-tree node with **m children:**

Number of keys = **m − 1**

Number of child pointers = **m**

$$(m-1) \times 8 + m \times 8 \leq 4096$$

$$8(2m-1) \leq 4096$$

$$2m - 1 \leq 512$$

$$m \leq 256$$

Max Keys and Children?

The node size is approximately (ignoring the small meta data components):

$$(m-1) \times \text{Key Size} + m \times \text{Pointer Size}$$

Why Large Order Makes B-trees Fast

Suppose you have 1 million records to search through.

A large order gives each node many more children, which dramatically reduces tree height. That matters because the main cost is **disk I/O**: fewer levels means fewer disk accesses.

Binary Search Tree	B-tree Order 256
Branching factor: **2**	Branching factor: **256**
Height: **~20**	Height: **~3**
Disk accesses: **20**	Disk accesses: **3**

The win is not just fewer comparisons , it is far fewer expensive disk reads.

INSERTION IN B-TREES

The insertion operation for a B Tree is done similar to the Binary Search Tree but the elements are inserted into the same node until the maximum keys are reached.

Add keys to the root in sorted order upto a max

Split when necessary. New nodes always added to the bottom level.

- Calculate the maximum keys (m−1) and, minimum ([m/2]−1) number of keys a node can hold, where m is denoted by the order of the B Tree.

Example: Let's create a B-tree with 5,3,21,9,13,22,7,10,11,14,8,16 where order is 4

Maximum Children = ?

Minimum Children = ?

Maximum keys = ?

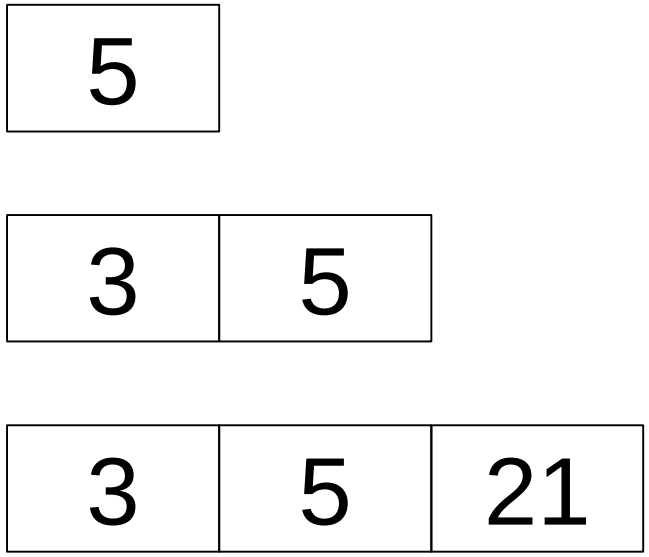
Minimum keys = ?

Maximum Children = 4
Minimum Children = 2
Maximum keys = 3
Minimum keys = 1

INSERTION IN B-TREES

- The data is inserted into the tree using the **binary search insertion** and once the **keys** reach the **maximum number,** the node is **split** into half and the **median key** becomes the **internal node** while the **left and right keys become its children.**

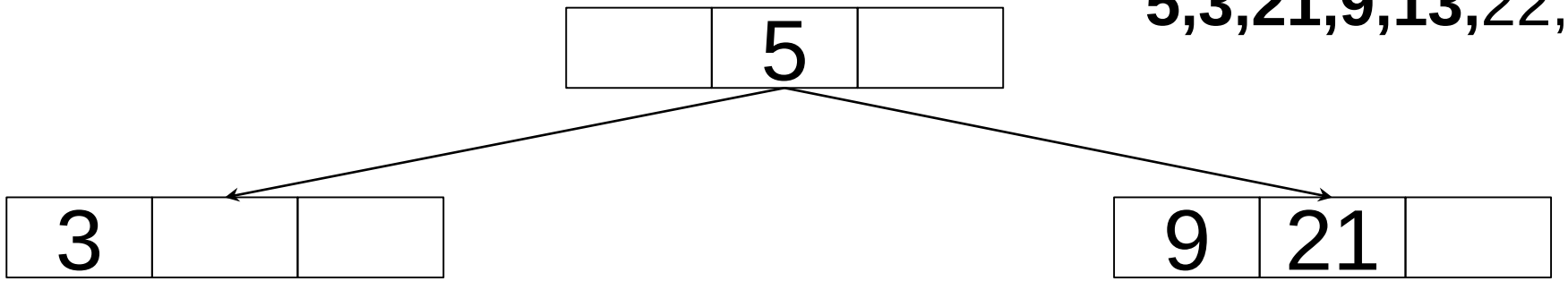
5,3,21,9,13,22,7,10,11,14,8,16

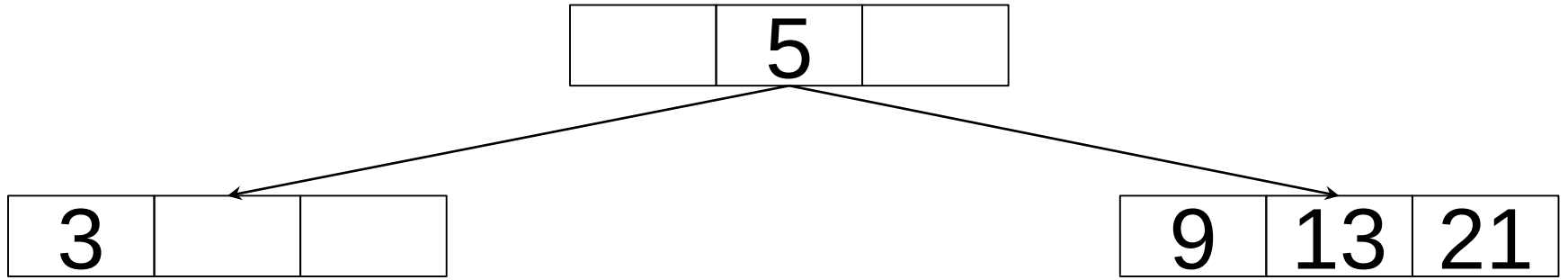


INSERTION IN B-TREES

- Next we have to insert 9, but adding 9 will cause overflow in the node, hence the node must be split.
- Given m keys (after inserting) in a full node: Promote key at index ⌊m/2⌋
  - for even number of keys, promote the middle-left key.
- All the leaf nodes should be on same level.

5,3,21,9,13,22,7,10,11,14,8,16



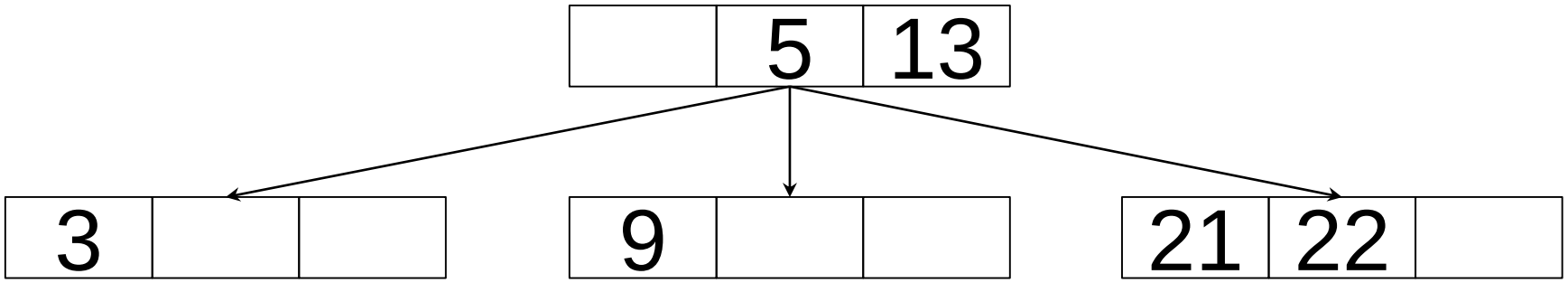


Maximum Children = 4
Minimum Children = 2
Maximum keys = 3
Minimum keys = 1

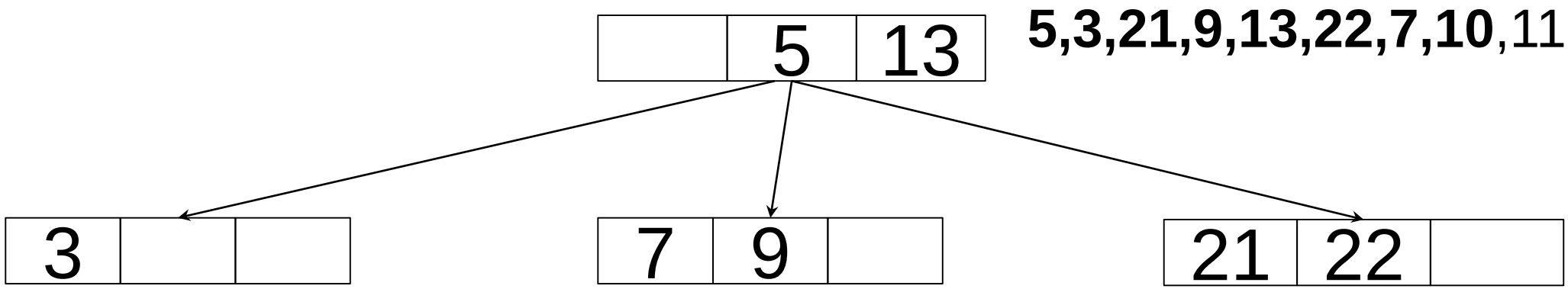
Maximum Children = 4
Minimum Children = 2
Maximum keys = 3
Minimum keys = 1

- The keys, 5, 3, 21, 9, 13 are all added into the node according to the binary search property but if we add the key 22, it will violate the maximum key property. Hence, the node is split in half, the median key is shifted to the parent node and the insertion is then continued.

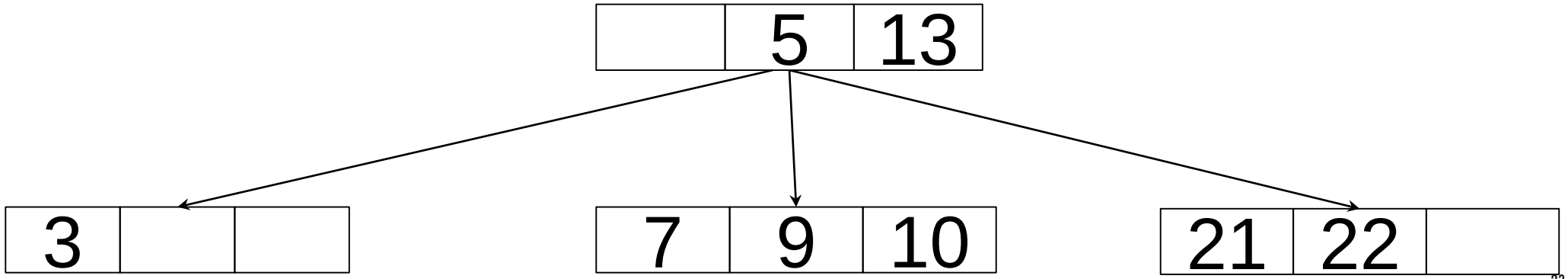
5,3,21,9,13,22,7,10,11,14,8,16



- Next 7,10 will be inserted

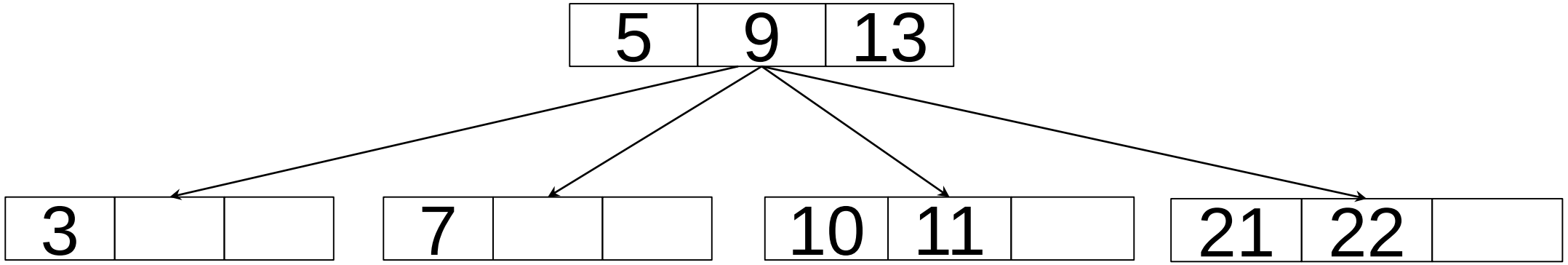


5,3,21,9,13,22,7,10,11,14,8,16



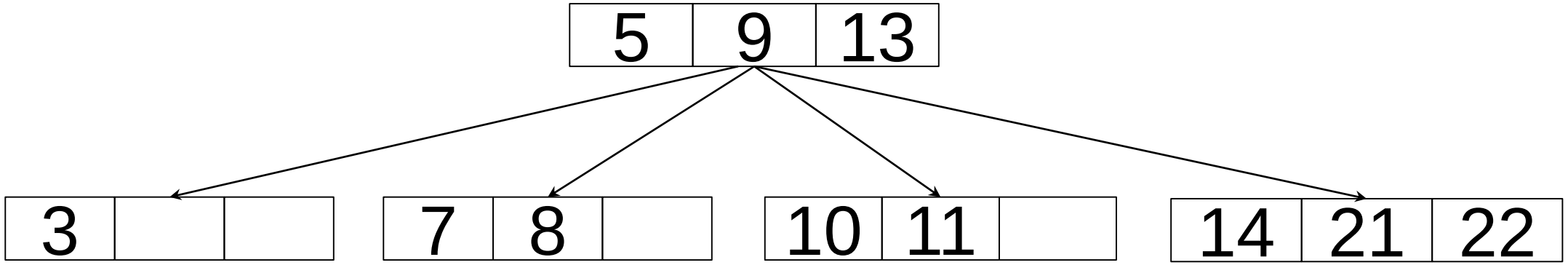
- Then we try to insert 11 which is greater than 5 and less than 13, so it should be in the middle node, but it will cause an overflow since all keys have been filled up. So the node is split and median is shifted to the parent

5,3,21,9,13,22,7,10,11,14,8,16



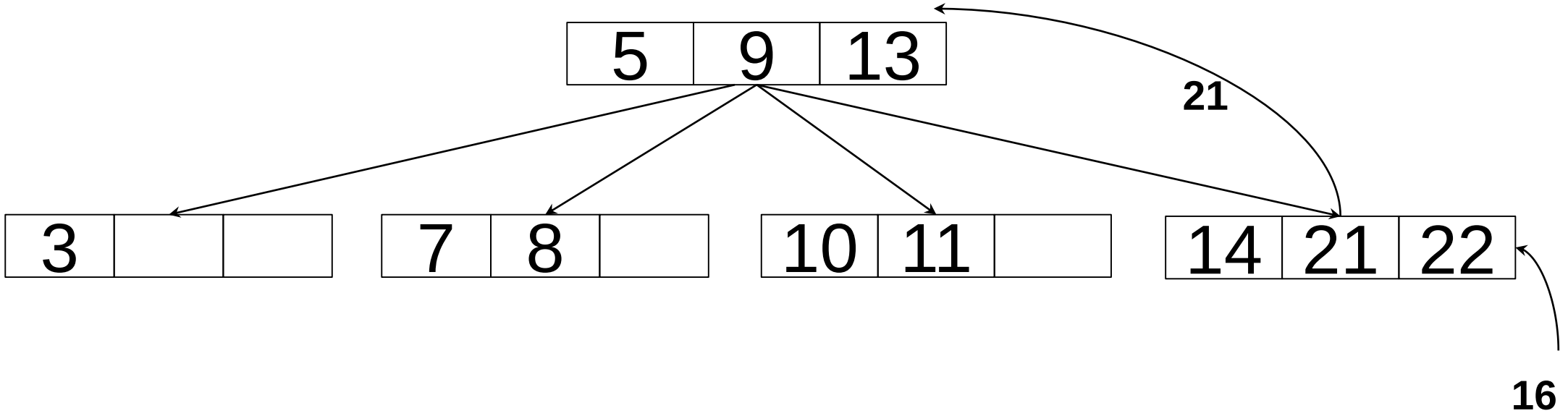
- Then we try to insert 14 and 8.

5,3,21,9,13,22,7,10,11,**14,8,**16

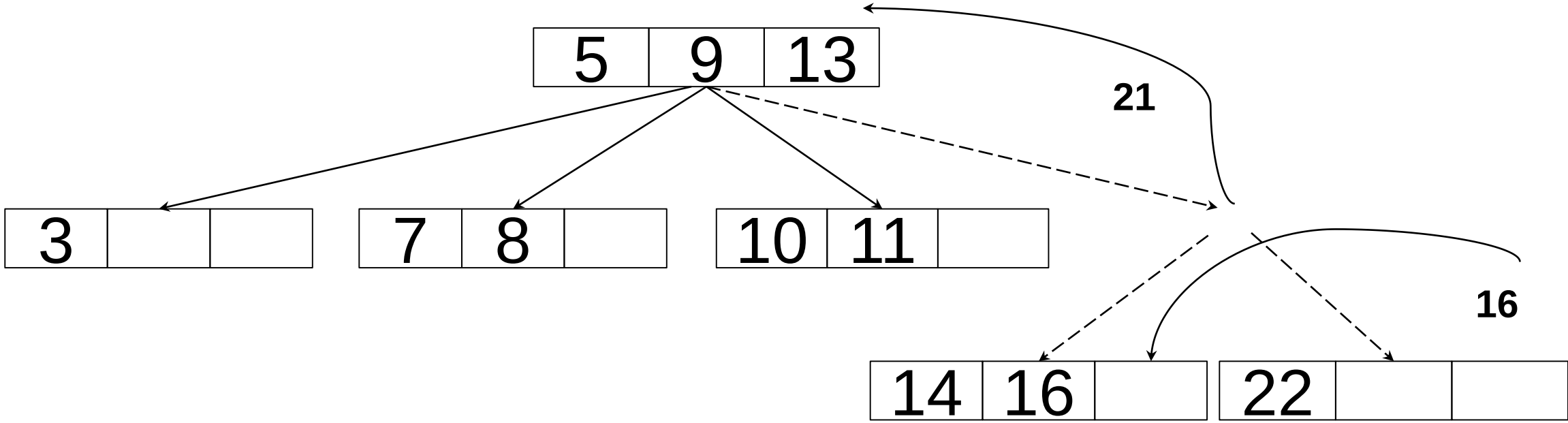


- While inserting 16, even if the node is split in two parts, the parent node also overflows as it reached the maximum keys. Hence, the parent node is split first and the median key becomes the root. Then, the leaf node is split in half the median of leaf node is shifted to its parent.

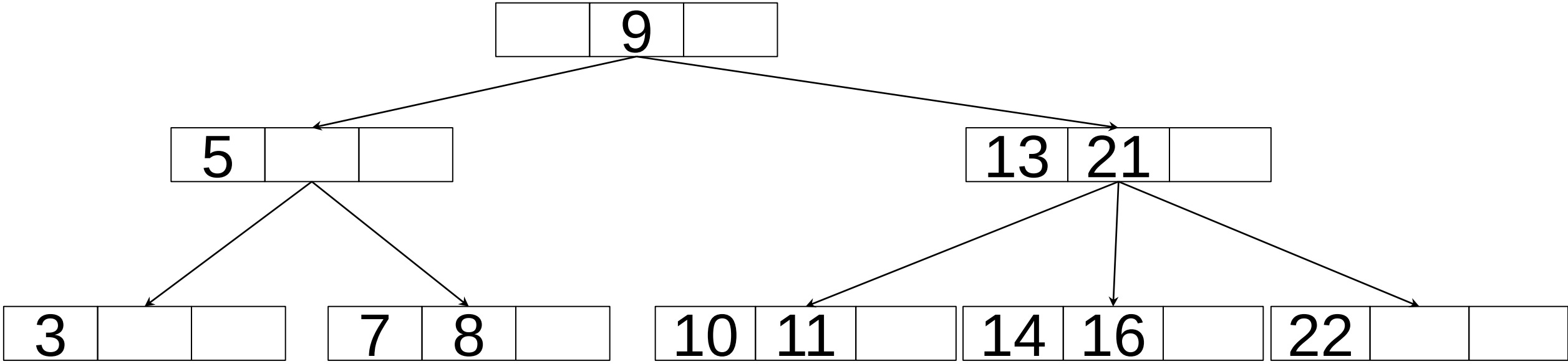
5,3,21,9,13,22,7,10,11,14,8,16



- Even the median is shifted to parent(which is the root here) the keys of it also filled, so again the root node is split in half, the median of the current root node becomes the new root and left and right keys will become new left and right child.



- With the split of previous root node needs to balance the previous children of the root node also



B-TREE INSERTION EXAMPLE

Order = 4

10,20,30,40,50,60,70,80,90,100

INSERTION IN B TREES FOR EVEN NUMBER OF KEYS

If there are even number of keys so split either go for left bias or right bias. You can choose one bias for entire procedure(you can not use both biasing on entire procedure).

Example: Let's create a Btree with 5,3,21,9,13,22,7,10,11,14,8,16 where order is 5

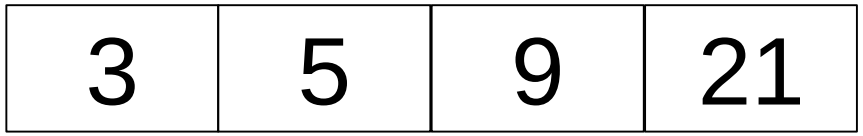
Maximum Children = 5

Minimum Children = 3

Maximum keys = 4

Minimum keys = 2

5,3,21,9,13,22,7,10,11,14,8,16

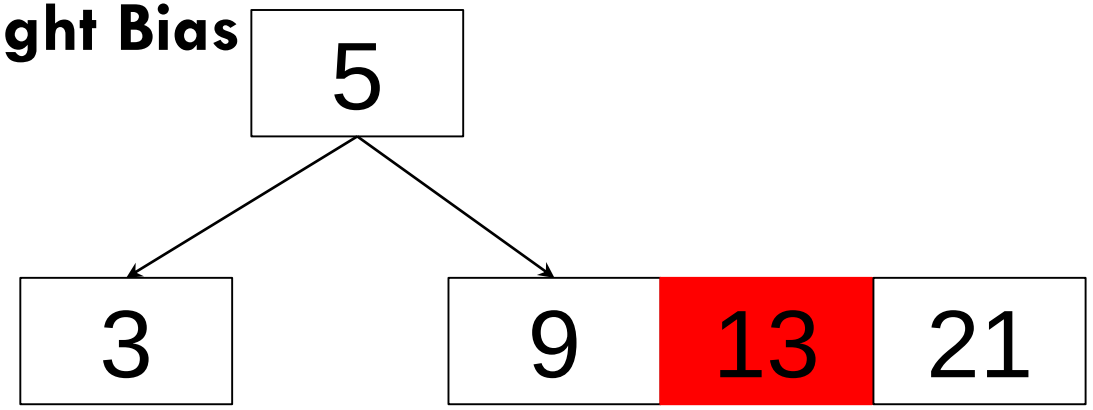


INSERTION IN B TREES FOR EVEN NUMBER OF KEYS

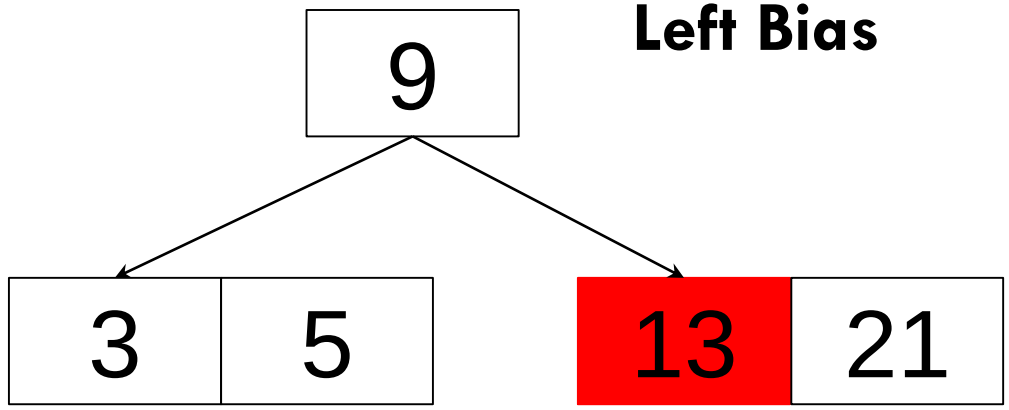
5,3,21,9,13,22,7,10,11,14,8,16

Neither split satisfies the B-tree property.

Right Bias



Left Bias



right-bias: The node is split such that its right subtree has more keys than the left subtree.
left-bias: The node is split such that its left subtree has more keys than the right subtree.

implementation-specific and is **not part of the standard B-tree definition.**

ACTIVITY 2

Perform the insertion operation for a B tree order 5 with following keys,

The elements to be inserted are

17,3,37,51,1,7,18,19,39,78,89,4,9,11,10

ACTIVITY 1

Perform the insertion operation for a B tree order 4 with following keys,

The elements to be inserted are

17,3,37,51,1,7,18,19,39,78,89,4,9,11,10

PROACTIVE SPLITTING

The proactive algorithm requires a node to have an **odd number of keys** when it is split.

Why?

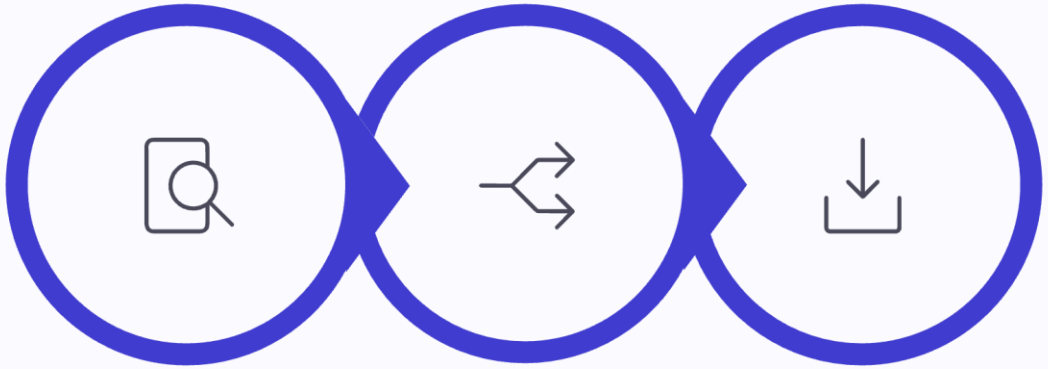
Because one key moves up, leaving an equal number of keys on both sides.

When Overflow Is Checked in B-Tree Insertion

Proactive Strategy

Split full nodes **before** inserting the key, so overflow never happens. minimum degree (t) is min no of children

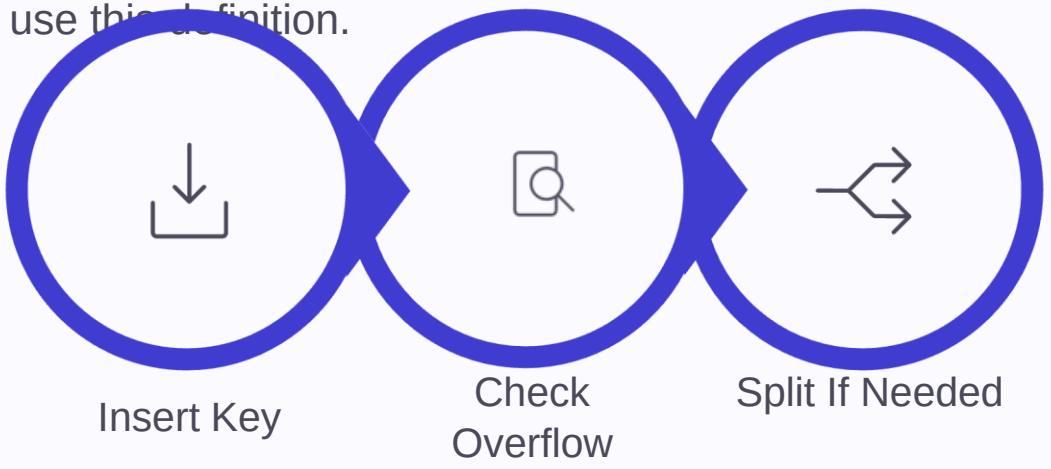
- Check whether the target node is full first
- If it has **2t-1 keys**, split it immediately
- Then insert the new key into a node with room
- **B-trees use this approach**



Reactive Strategy

Insert the key first, then handle overflow only if the node becomes full.

- Place the key in the target node
- If the node overflows, split it afterward
- Overflow is detected **after** insertion
- **B-trees use this approach**, many algorithm textbooks use this definition.



- **Proactive:** split first, then insert
- **Reactive:** insert first, then split if needed

Minimum Degree *t* Calculations

B-tree node limits are determined directly by the minimum degree *t*. The formulas below show the key constraints at a glance.

Maximum children

$2t$

Minimum children

t

Maximum keys

$2t-1$

Minimum keys

$t-1$

Example with *t=3*: maximum children = 6, minimum children = 3, maximum keys = 5, minimum keys = 2.

What this means: as *t* grows, each node can hold more children and more keys, which keeps the tree shallow.

INSERTION ALGORITHM

```
If the tree is empty:
    Allocate a new root node 'root'.
    Insert 'key' into 'root'.
    Set 'tree'.root to 'root'.
    Return.


Set 'current' to 'tree'.root.

While 'current' is not null:
    a. If 'current' is a leaf node:
        i. Insert 'key' into 'current' in sorted order.
        ii. If 'current' has more keys than allowed:
            - Invoke SplitNode('current').
            - If 'current' was the root, update 'tree'.root.
        iii. Return.
    b. Otherwise:
        i. Determine the child node 'child' that should contain 'key'.
        ii. If 'child' is full:
            - Invoke SplitNode('child').
            - Update 'current' to reflect the split.(Adjust the Parent Node's Child References)
        iii. Set 'current' to 'child'.
```

CONTINUED.

```
Procedure SplitNode(node):
    If the number of keys in 'node' is odd, 'median' is the middle key.
    If the number of keys is even, 'median' is one of the two middle keys (right-bias or left-bias).
    Create two new nodes 'newLeft' and 'newRight'.
    If 'node' is the root:
        a. Create a new root with 'median'.
        b. Set the new root's children to 'newLeft' and 'newRight'.
        c. Update the tree's root pointer.
    Else (if 'node' is not the root):
        a. Move 'median' up to the parent node in sorted order.
        b. If moving 'median' causes the parent node to overflow,
        - Recursively apply SplitNode to the parent.
        c. Update parent's child pointers to point to 'newLeft' and 'newRight'.
    Distribute keys and children:
        Move keys less than 'median' to 'newLeft'.
        Move keys greater than 'median' to 'newRight'.
        If 'node' has children, distribute them between 'newLeft' and 'newRight'.
```

B-TREE- DELETE KEYS

Find the key needs to be deleted

Delete it.

What issues are possible?

If the nodes is less than min when deleted.

1. Get from the sibling nodes
2. Merge siblings if they are also at min

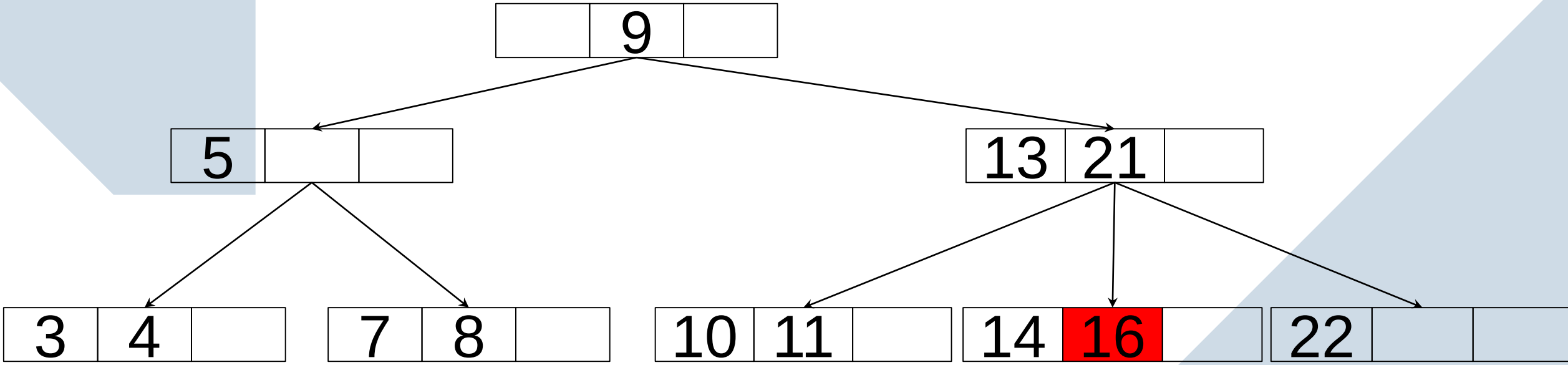
UCSC

Deletion in B-Trees

Maximum Children = 4
Minimum Children = 2
Maximum keys = 3
Minimum keys = 1

The deletion operation in a B tree is slightly different from the deletion operation of a Binary Search Tree.

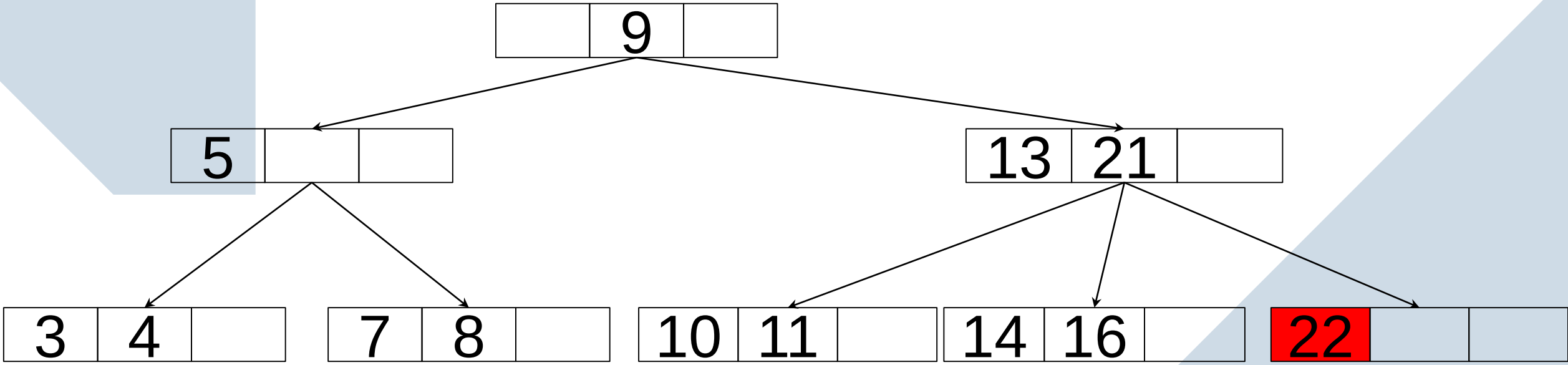
- **Case 1** – If the key to be deleted is in a leaf node and the deletion does not violate the minimum key property, just delete the key.



Deletion in B-Trees

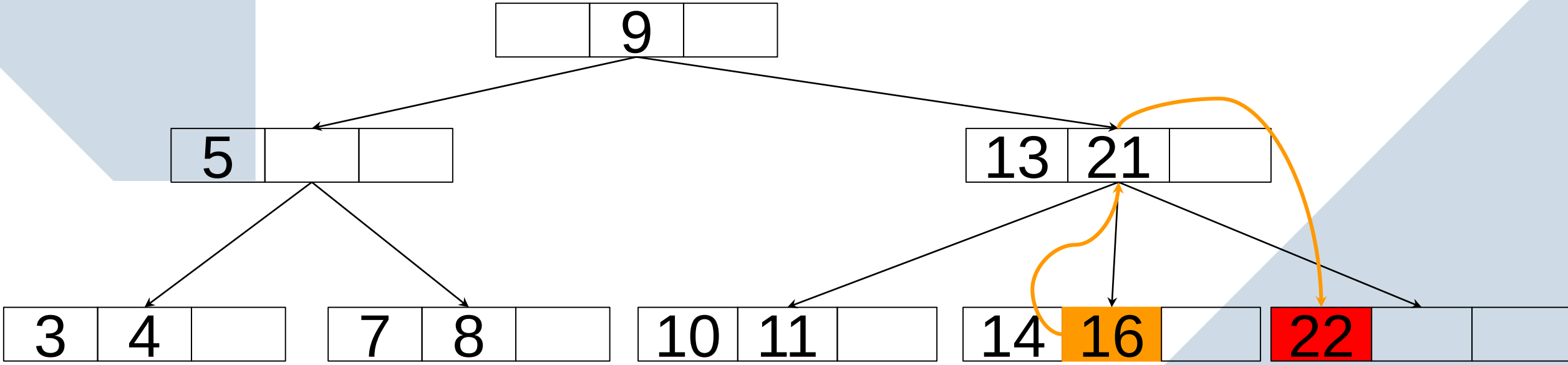
Maximum Children = 4
Minimum Children = 2
Maximum keys = 3
Minimum keys = 1

- **Case 2** – If the key to be deleted is in a leaf node but the deletion violates the minimum key property, borrow a key from either its left sibling or right sibling.

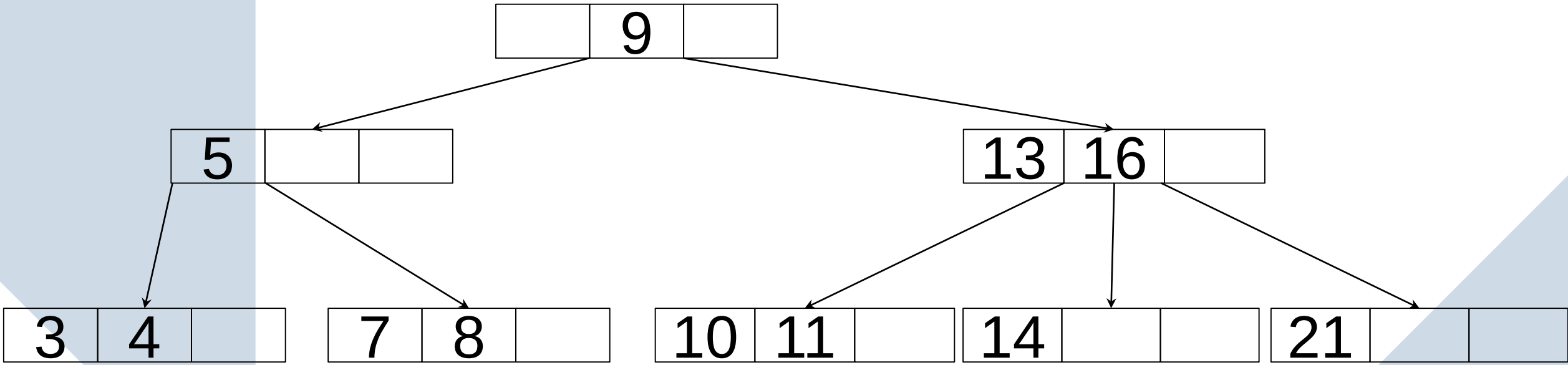


Deletion in B-Trees

Case 2 contd: If I borrow a key from left node, it will be alright since that node has two keys. So first we have the move max key of left sibling to parent node and parent key to target node and then delete target key

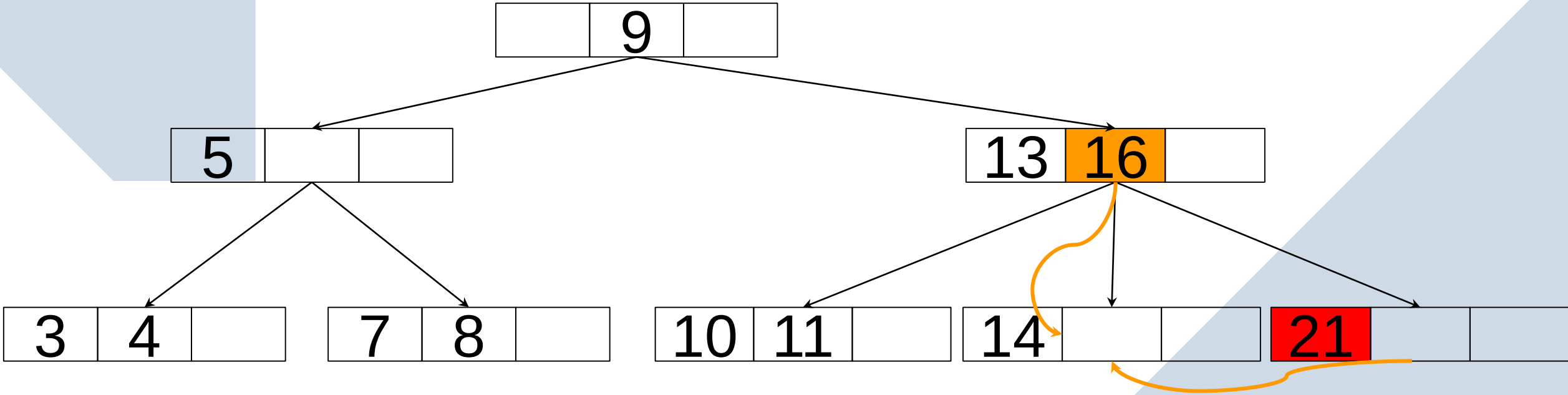


Deletion in B-Trees



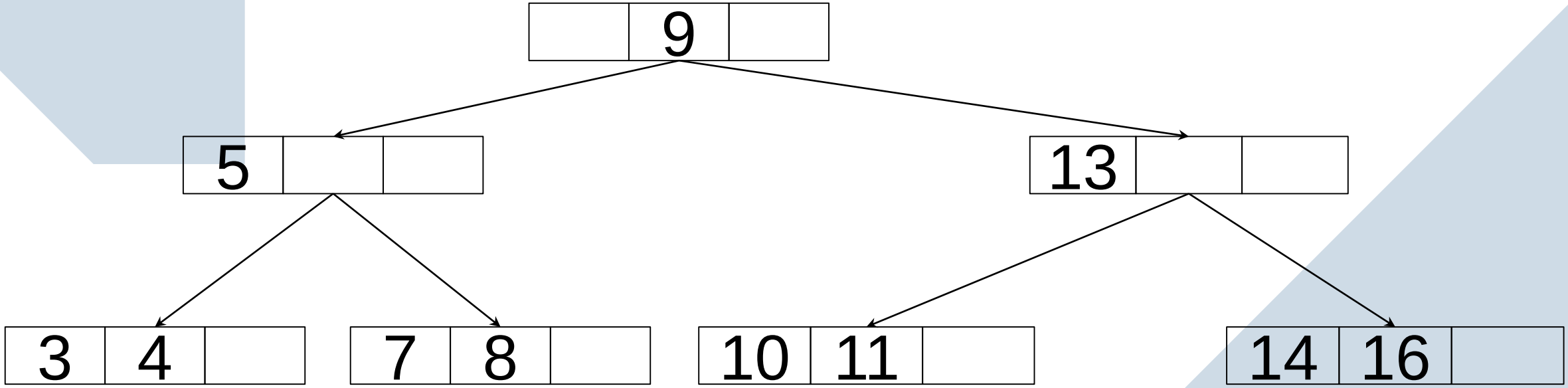
Deletion in B-Trees

Case 3: In this case both left and right siblings contain min no of keys, so we cannot borrow from either of them. Both the above case fails, then we can merge target node with either with left or right sibling along with parent key then we can delete target key.



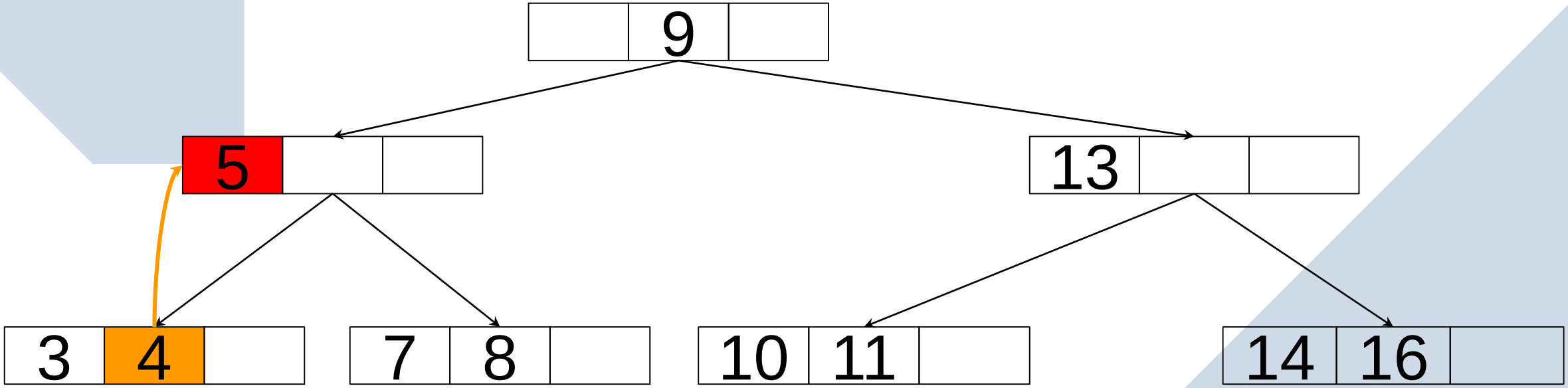
Deletion in B-Trees

Case 3 Contd: While merging parent key 16 if there is less no. of key left than min no. of key in parent node. Suppose only key was 16 in the parent node then we have to borrow from the neighbor 5, but if the neighbor have min no. of keys then we have to repeat the step by merging childs with the parent node



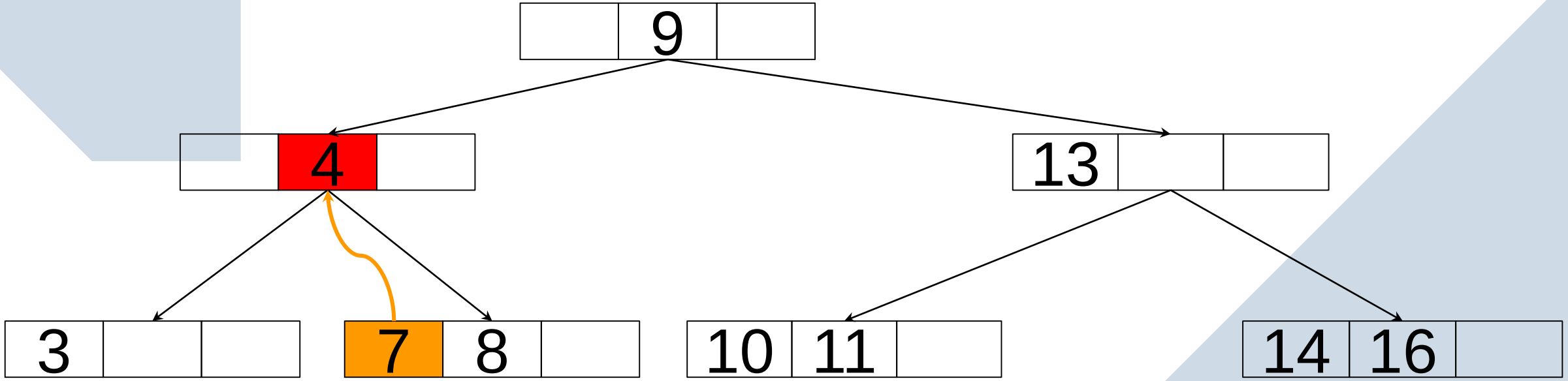
Deletion in B-Trees

Case 4: Deletion of an internal node key, In this case we replace the key with its inorder predecessor or successor because predecessor node have more than min no. of key, and recursively delete the predecessor: (i.e. *4*)



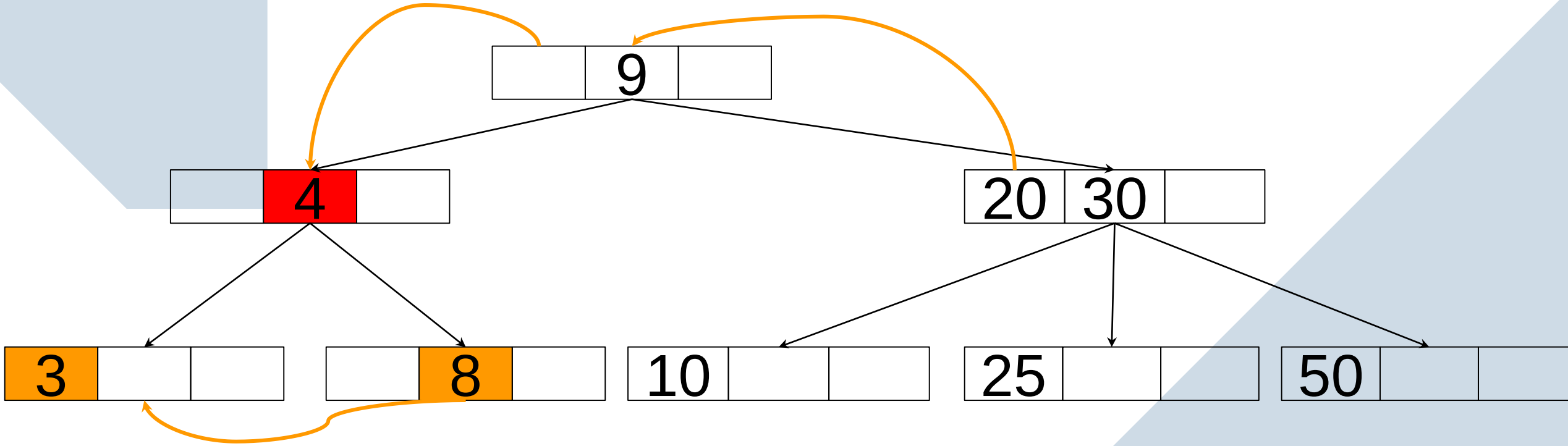
Deletion in B-Trees

Case 4.1: Repeat the step and delete another internal node and replace it with successor.



Deletion in B-Trees

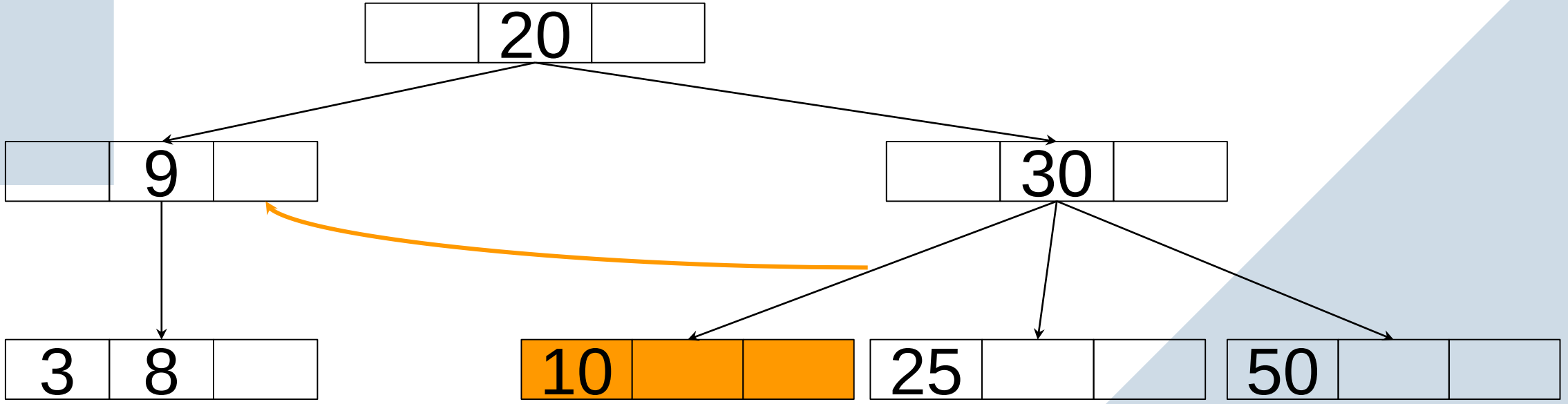
Case 5: Delete an internal node when their successor and predecessor have minimum number of nodes, so we cannot replace with either of them. In this case we merge the both the predecessor and successor node and then check if we can borrow from the sibling node, since the sibling node have more than minimum keys we can borrow from it.



Deletion in B-Trees

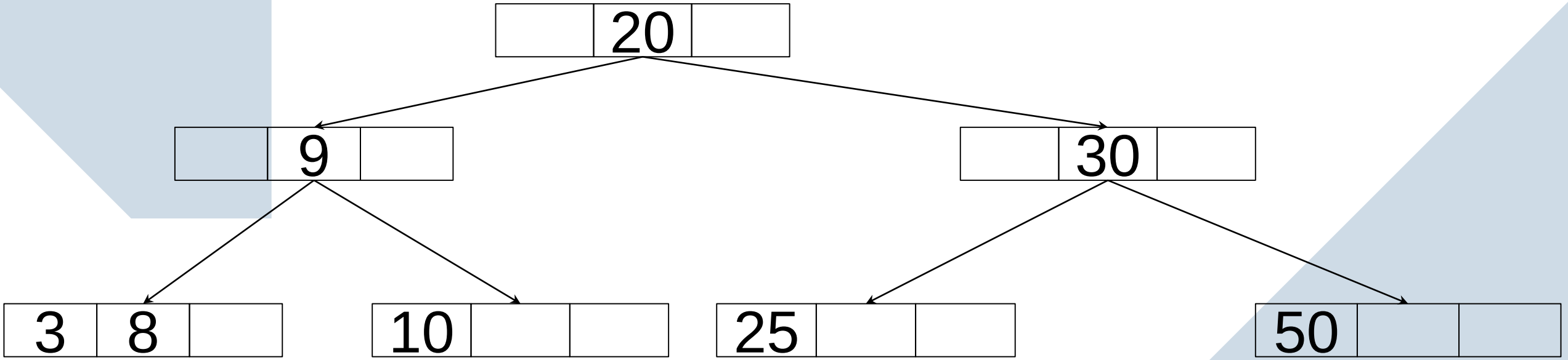
Maximum Children = 4
Minimum Children = 2
Maximum keys = 3
Minimum keys = 1

Case 5 Contd: After borrowing a key from the sibling, ensure that the target node still has enough child nodes. If not, you may need to perform further adjustments.



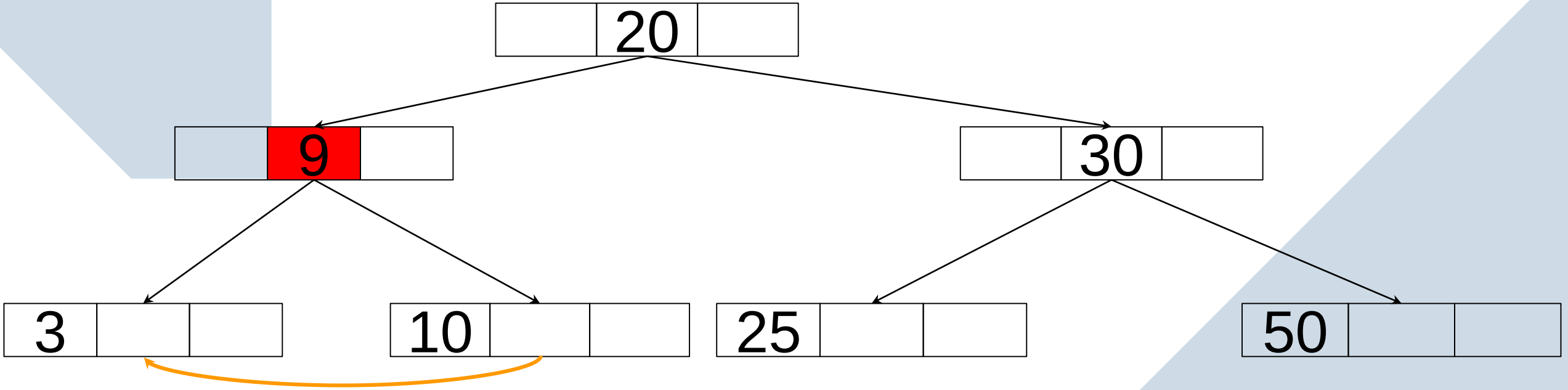
Deletion in B-Trees

Case 5 Contd: Since borrowed from the right sibling leftmost child becomes right child of the target node.



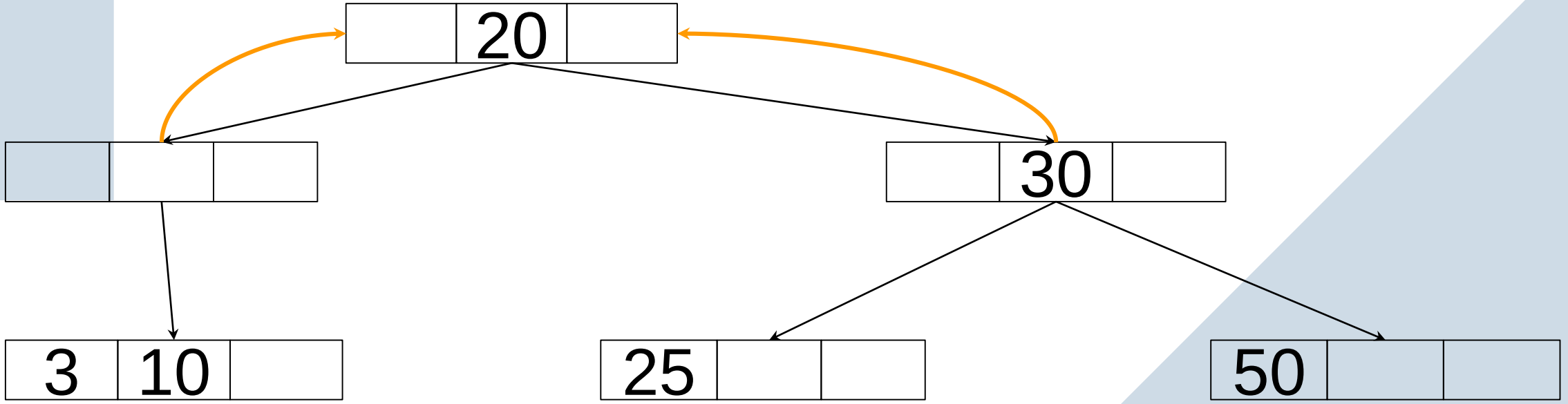
Deletion in B-Trees

Case 5.1: Delete an internal node when their successor and predecessor have minimum number of nodes, and sibling also have minimum number of nodes. We can merge the both the predecessor and successor node

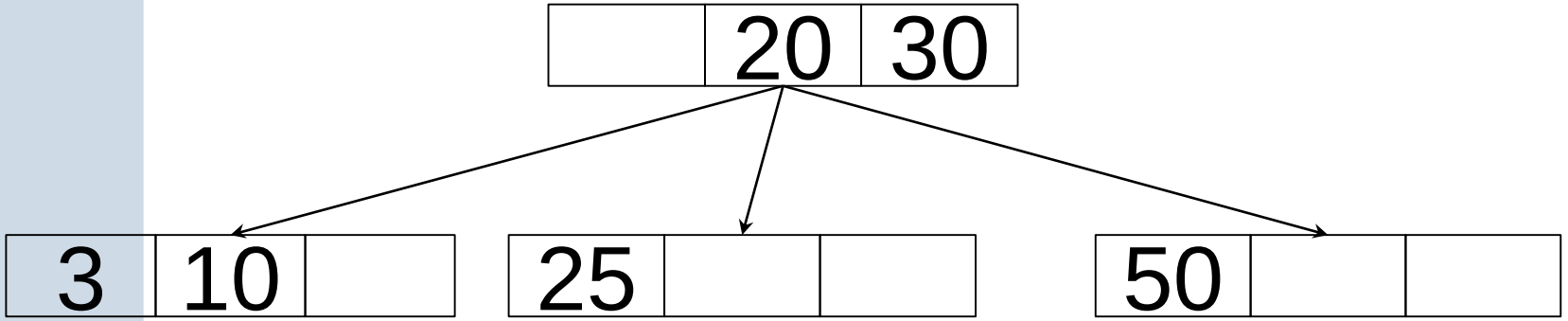


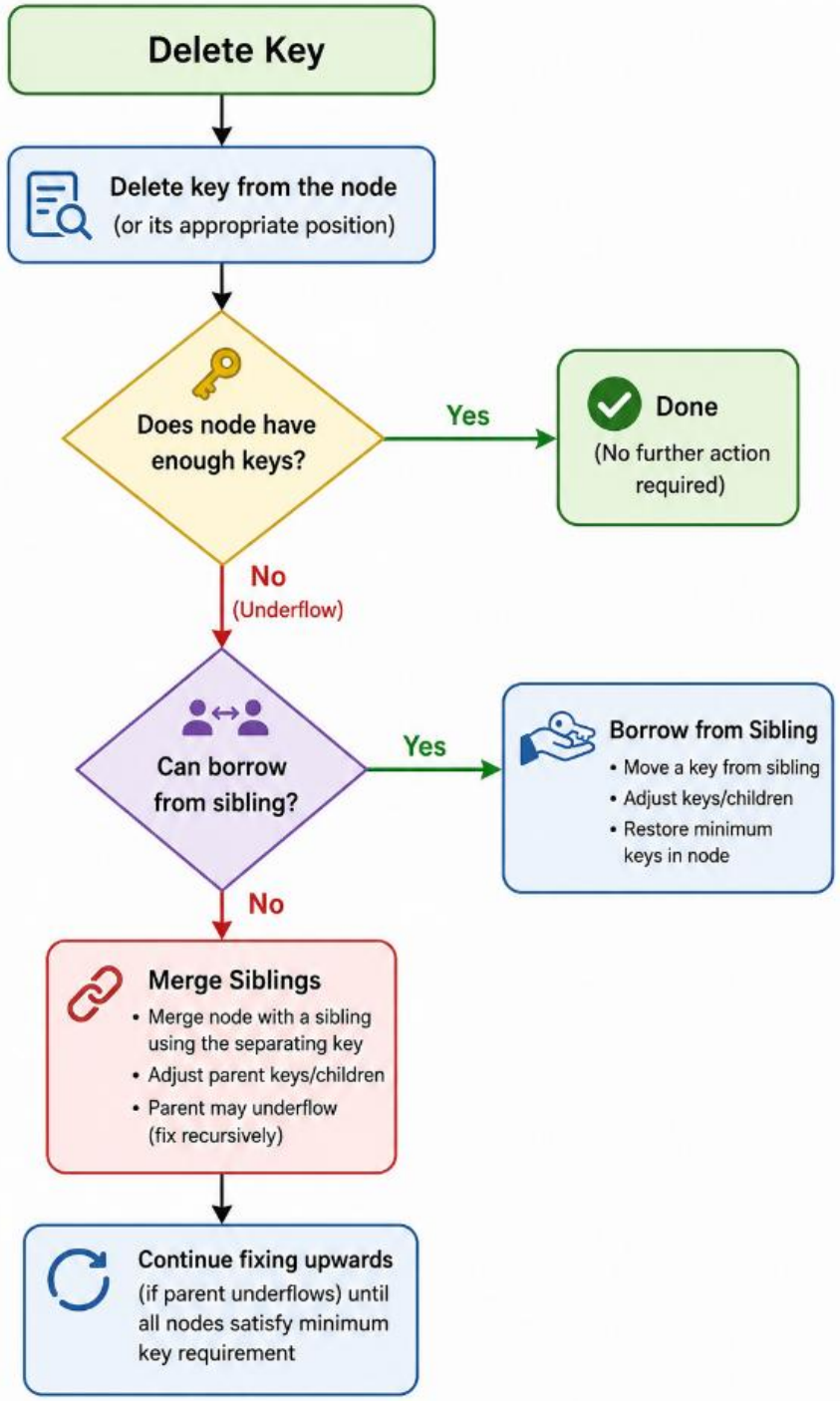
Deletion in B-Trees

Case 5.1 Contd: Now the problem occurs when we try to delete 9 because now this node have less than min no of keys. Which invalid for this b-tree, so merge the target node with sibling along with the parent node.



Deletion in B-Trees



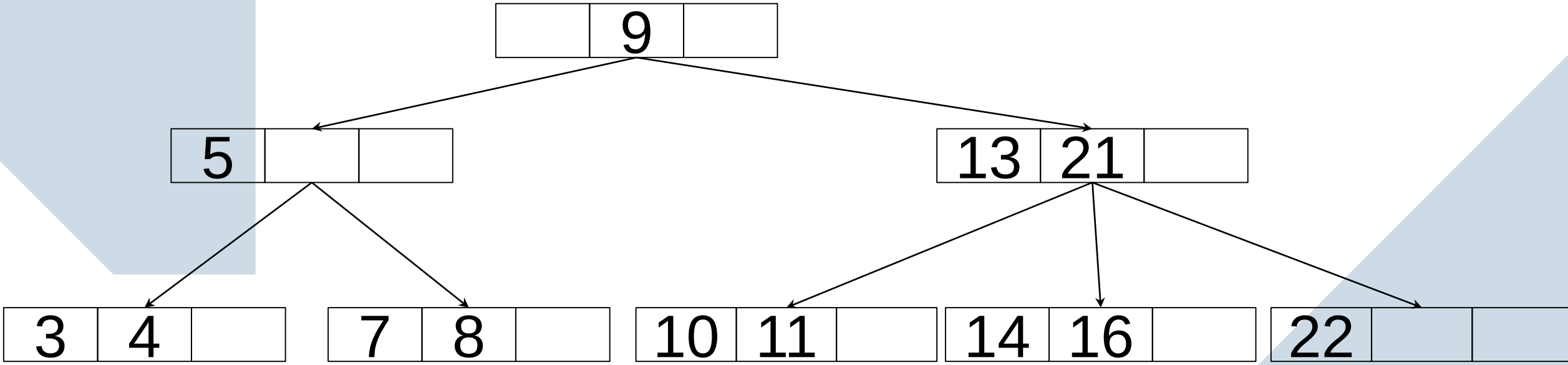


Borrow when a sibling has a spare key.

Merge when all siblings are already at minimum capacity.

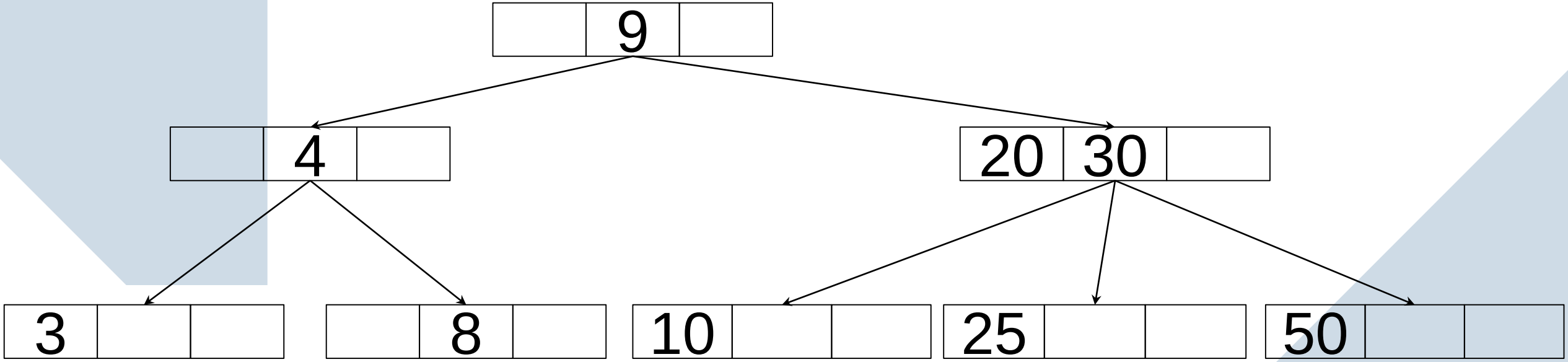
Activity 1

Delete the keys 16, 22, 21, 5, 4 from the below tree



Activity 2

Delete the keys 4, 8, 9 from the below tree



DELETION ALGORITHM — B - TREE

Terminology

- `t` : minimum degree = ceil(m / 2)
- Each node can contain `t - 1` to `m - 1` keys (except the root).
- A full node has `m - 1` keys.
- A non-root node must have at least `t - 1` keys.
- Internal node: has children
- Leaf node: no children

Deletion Algorithm

```
If the tree is empty:
    Return.

Set 'current' to the root of the tree.
Search for the node containing the key to be deleted.
```

```
WHILE 'current' is not null:
    IF 'current' is a leaf node:
        IF the key is in 'current':
            Delete the key from 'current'.
            CALL Post-Deletion Balancing with 'current'.
        RETURN; // Deletion completed
    ELSE: // 'current' is an internal node
        IF the key is in 'current':
            // Replace the key with in-order predecessor or successor
            FIND the in-order predecessor 'pred' (max key in the left subtree).
            FIND the in-order successor 'succ' (min key in the right subtree).

            IF 'pred' exists and has more than the minimum keys:
                REPLACE the key in 'current' with the maximum key from 'pred'.
                SET 'current' to 'pred' and continue deletion.
            ELSE IF 'succ' exists and has more than the minimum keys:
                REPLACE the key in 'current' with the minimum key from 'succ'.
                SET 'current' to 'succ' and continue deletion.
            ELSE:
                // Merge 'current' with one of its children and the parent key
                Merge 'current' with a sibling using a parent key.
                CALL Post-Deletion Balancing on the merged node.
        ELSE:
            // Move to the child that should contain the key
            SET 'current' to the appropriate child.
```

Deletion Algorithm

```
Post-Deletion Balancing:
    IF 'node' has fewer keys than the minimum required:
        WHILE 'node' has fewer keys than the minimum required:
            FIND the immediate sibling of 'node' with more than the minimum keys.
            IF such a sibling exists:
                CALL BorrowKey('node', sibling).
                RETURN; // Node is balanced
            ELSE:
                CALL MergeNodes('node').
                SET 'node' to the parent; // Continue balancing up the tree if necessary
    ELSE:
        RETURN; // Node is balanced
```

```
Borrow Key Process:

    IF 'isLeftSibling':
        BORROW the rightmost key from the left sibling, moving it to the parent.
        MOVE the parent's key to 'node'.
        IF nodes are internal:
            The rightmost child of the left sibling becomes the leftmost child of 'node'.

    ELSE:
        BORROW the leftmost key from the right sibling, moving it to the parent.
        MOVE the parent's key to 'node'.
        IF nodes are internal:
            The leftmost child of the right sibling becomes the rightmost child of 'node'.

    UPDATE all pointers and parent's keys as necessary.

Merge Nodes Process:
    MERGE 'node' with its sibling using a key from the parent.
    COMBINE keys and children of 'node', sibling, and the parent's key into one node.

    IF nodes are internal:
        Merge the successor and predecessor.

    UPDATE the parent's child pointers and remove the merged key.

    IF the parent is the root:
        SET the merged node as the new root.
```

DELETION ALGORITHM – B - TREE

Main Function

```
BTreeDelete(T, k):
    DeleteKey(T.root, k)
    if T.root has 0 keys and is not a leaf:
        T.root = T.root.children[0]
```

DELETION ALGORITHM — B - TREE

Recursive Deletion Function

```
DeleteKey(x, k):
    i = FindIndex(x, k)

    if i < x.n and x.keys[i] == k:       // Case 1 or 2
        if x is leaf:
            Remove k from x              // Case 1: delete directly
        else:
            if x.children[i] has ≥ t keys:
                pred = Predecessor(x.children[i])
                x.keys[i] = pred
                DeleteKey(x.children[i], pred)
            else if x.children[i+1] has ≥ t keys:
                succ = Successor(x.children[i+1])
                x.keys[i] = succ
                DeleteKey(x.children[i+1], succ)
            else:
                Merge(x, i)
                DeleteKey(x.children[i], k)
    else:                                // Case 3: key not found in x
        if x is leaf:
            return "Key not found"
        flag = (i == x.n)
        if x.children[i] has only t - 1 keys:
            FixChild(x, i)
        if flag and i > x.n:
            DeleteKey(x.children[i-1], k)
        else:
            DeleteKey(x.children[i], k)
```

DELETION ALGORITHM — B - TREE

FixChild(x, i)

```
FixChild(x, i):
    if i > 0 and x.children[i-1] has ≥ t keys:
        BorrowFromPrev(x, i)
    else if i < x.n and x.children[i+1] has ≥ t keys:
        BorrowFromNext(x, i)
    else:
        if i < x.n:
            Merge(x, i)
        else:
            Merge(x, i-1)
```

Merge(x, i)plaintextCopyEdit

```
Merge(x, i):
    y = x.children[i]
    z = x.children[i+1]
    y.keys.append(x.keys[i])
    y.keys += z.keys
    if not y is leaf:
        y.children += z.children
    x.keys.remove(i)
    x.children.remove(z)
```

DELETION ALGORITHM – B - TREE

BorrowFromPrev(x, i)

```
BorrowFromPrev(x, i):
    child = x.children[i]
    sibling = x.children[i-1]
    child.keys.insert(0, x.keys[i-1])
    if not child is leaf:
        child.children.insert(0, sibling.children.pop())
    x.keys[i-1] = sibling.keys.pop()
```

BorrowFromNext(x, i)

```
BorrowFromNext(x, i):
    child = x.children[i]
    sibling = x.children[i+1]
    child.keys.append(x.keys[i])
    if not child is leaf:
        child.children.append(sibling.children.pop(0))
    x.keys[i] = sibling.keys.pop(0)
```

```
FindIndex(x, k):
    i = 0
    while i < x.n and k > x.keys[i]:
        i = i + 1
    return i
```

SEARCHING A KEY

1. Search the keys inside the current node.
2. If the key is found → return success.
3. If the key is not found → decide which child subtree can contain the key.
4. Repeat until the key is found or a leaf node is reached.

B-TREE- TIME COMPLEXITY

Sr. No.	Operation	Time Complexity
1.	Search	O(log n)
2.	Insert	O(log n)
3.	Delete	O(log n)
4.	Traverse	O(n)

UCSC

Advantages of B Tree

- **Balanced Tree Structure**: B-trees are self-balancing, which ensures that the height of the tree remains logarithmic with respect to the number of keys, guaranteeing efficient operations.
- **Search, Insert, Delete Operations:** All these operations can be done in O(logn) time
- **Good for Large Datasets:** B-trees can handle an enormous amount of data because they grow in width, not in height. This makes them suitable for databases that contain millions or billions of records.
- **Efficient Disk Access:** B-trees have a high branching factor, meaning that each node can have many children. Since each node of a B-tree can store multiple keys (and therefore multiple records or pointers to records), fewer nodes need to be accessed to find any given key

Applications of B Tree

- **Databases:** B-trees are extensively used in database systems to manage large datasets by indexing them, which allows for quick search, insert, delete, and range queries.
- **File Systems:** Many file systems use B-trees for metadata management, enabling fast file lookups, directory traversals, and space allocation.
- **Main Memory Storage:** With B-trees, it is efficient to maintain a large amount of data in the main memory for applications requiring quick access.
- **Caching:** B-trees are used in cache-conscious applications that require efficient search and modifications while minimizing cache misses.

EXAMPLE DB INDEX

When to Use B-tree Indexes

1. Primary Key Indexes: B-trees are often used for the primary key indexes because the data is unique and frequently queried.

2. Non-unique Indexes: B-trees are also efficient for secondary indexes, where multiple rows might have the same value for the indexed column (e.g., searching by department).

3. Range Queries: B-trees are particularly well-suited for range queries (BETWEEN, >=, <=) because the keys are stored in sorted order, and the tree can quickly find the range of values.

```
CREATE INDEX idx_employee_id ON Employee(Employee_ID);
```

UCSC

THANK YOU !

I am available at : ysr@ucsc.cmb.ac.lk

UCSC